Computers & Security 170 (2026) 105018

Contents lists available at ScienceDirect

Computers & Security

journal homepage: www.elsevier.com/locate/cose

Uncovering robot joint-level controller actions from encrypted network traffic: Empirical attacks and information-theoretic bounds☆

Cheng Tang, Diogo Barradas, Urs Hengartner, Yue Hu *

University of Waterloo, 200 University Ave W, Waterloo, N2L 3G1, Ontario, Canada

ARTICLE INFO

Keywords:
Encrypted traffic analysis
Privacy
Robot teleoperation
Information theory
Traffic fingerprinting

ABSTRACT

This study examines the privacy risks associated with the teleoperation of robots controlled via encrypted network communications. From the perspective of a network eavesdropper, we explore the potential to infer sensitive robotic actions by analyzing traffic metadata, such as packet timing, size, and direction. We investigate this threat using a smartphone's Inertial Measurement Unit (IMU) to control a collaborative robotic arm via three joint-level modalities — position, velocity, and torque — to perform four distinct actions. First, empirical traffic analysis demonstrates that an adversary can identify robot actions with high accuracy using standard machine learning classifiers. Second, to determine whether the remaining classification errors stem from empirical model limitations or the structural constraints of the physical protocols, we apply a classifier-agnostic information-theoretic evaluation. Using mutual information, we derive Bayes optimal accuracy bounds and show that torque control leaks more deterministic information than suggested by empirical models, while velocity inherently exposes less information. Third, by prototyping a traffic padding defense, we evaluate the limitations of standard obfuscation against these structural leakages. Our findings highlight the necessity of information-theoretic bounds in privacy research and inform the design of secure robotic teleoperation APIs.

1. Introduction

Current research is pushing towards the adoption of teleoperated robots, particularly collaborative robot arms, in everyday settings, including homes (Cabrera et al., 2021) and production lines (González et al., 2021), thanks to their versatility and usability for diverse tasks. In the teleoperation setting, the robot and the controller (i.e., the computer on which the control software is run) are typically connected to a network, often the internet (Romeo et al., 2020; DeMarinis et al., 2019).

The network traffic exchanged between the controller and the robot are typically encrypted for preventing a network adversary from reading or tampering with the commands sent by the controller. However, despite these safeguards, observers can still try to infer the content of encrypted messages by analyzing communication metadata, such as the size, direction, and timing of network packets. For instance, inference techniques have previously succeeded in identifying the websites users visit through encrypted tunnels (Hayes and Danezis, 2016; Sirinam et al., 2018) that aim to hide one's browsing patterns. However, this threat remains underexplored in robot teleoperation scenarios. Robotics research has traditionally focused on ensuring low latency, reliability, and control performance, often assuming trusted communication environments, while privacy risks from traffic metadata have received limited attention. Additionally, the domain-specific communication stacks used in robotics (e.g., Robotic Operating System — ROS, Data Distribution Service — DDS, or proprietary interfaces) have been less studied by the traffic analysis and cybersecurity communities compared to mainstream internet applications. If similar inference techniques were as effective on encrypted robot/controller traffic as on web traffic, adversaries could gain unauthorized access to private information which, in sensitive settings like home care, could reveal information about an individual's care regimen and health status (such as medication dispensing or physical therapy).

The *first objective* of our research is to showcase the practical feasibility of inferring robot actions from encrypted network traffic patterns. We leverage an intrinsic property of robotic teleoperation traffic: the unique network traffic pattern generated by each command and its associated feedback. For example, the completion time of a motion can be reflected by observing the frequency between network packets with specific lengths. Even with encryption, certain characteristics of

☆ This article is part of a Special issue entitled: 'IFIP SEC 2025' published in Computers & Security.

* Corresponding author.
E-mail addresses: c225tang@uwaterloo.ca (C. Tang), dbarrada@uwaterloo.ca (D. Barradas), urs.hengartner@uwaterloo.ca (U. Hengartner), yue.hu@uwaterloo.ca (Y. Hu).

https://doi.org/10.1016/j.cose.2026.105018
Received 9 March 2026; Received in revised form 24 May 2026; Accepted 8 June 2026
Available online 11 June 2026
0167-4048/© 2026 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

this traffic pattern, such as the timing between packets, may allow an observer to differentiate between commands.

To experiment with these inference attacks, we employ a smartphone as the teleoperation interface (Wu et al., 2020; Sankar et al., 2023) (similarly to Rodriguez et al. (2017)). We use the phone's Inertial Measurement Unit (IMU)'s gyroscope and accelerometer to control a collaborative robot arm. The smartphone is connected to a computer that is connected to the same network as the robot and commands the motions to the robot via three types of joint-level controllers: position, velocity, and torque. For instance, the end-effector (EE) of the robot arm follows the movements of the smartphone — if the smartphone is moved up and down, the robot arm's EE also moves up and down. For robot arms, all control strategies essentially boil down to joint-level controllers, which set specific values (e.g., positions, velocities, or torques) at joint level depending on the desired outcome. For instance, torque control might be more desirable when physical interactions with human users may occur, while position control might be prioritized for precision in movement.

The *second objective* of our research is to identify how different control strategies impact the privacy of teleoperated robotic systems. We posit that different joint-level controllers influence the type and granularity of data inferable from encrypted network traffic. For example, a controller adjusting joint positions predictably may make action inference easier from network metadata, compared to a torque-based control scheme that produces more variable traffic patterns. To the best of our knowledge, we are the first to study the prediction of robot actions from encrypted joint-level controller traffic. In our experiments, we execute four distinct movements 100 times each using each of the three controllers and collect the encrypted network traffic transferred between the computer and the robot. We analyze different features of this network traffic, using them to train and evaluate different machine learning classifiers.

The *third objective* of our research is to establish theoretical bounds on the privacy leakage of teleoperated robotic systems by applying information-theoretic analysis to encrypted robot network traffic. While empirical machine learning classifiers can demonstrate the feasibility of traffic inference attacks, their accuracy may be limited by model design, feature engineering, or training data. To determine the fundamental inference capability available to an adversary, we leverage WeFDE (Li et al., 2018), an information-theoretic framework originally developed for website fingerprinting. By adapting this established methodology to the novel use case of robotics fingerprinting, we estimate the mutual information between observable traffic features and robotic actions to derive Bayes optimal accuracy bounds. This classifier-agnostic analysis allows us to characterize the intrinsic information leakage of teleoperation protocols independently of the limitations of specific machine learning models.

Contributions. We deliver the following main contributions:

- We investigate encrypted network traffic patterns from various angles, including the effects of command frequency (i.e., the frequency at which the robot is requested to perform a movement as part of an action) on packet sizes and arrival times, as well as the time it takes to complete commands.
- We assess the effectiveness of several traffic pattern recognition methods and identify reliable techniques for deducing robot actions from encrypted traffic.
- We adapt an existing information-theoretic framework to the novel domain of robotic teleoperation. By analyzing encrypted network traffic, we quantify intrinsic privacy leakage and derive Bayes optimal bounds on action inference.
- We explore the consequences of our research for the design of new privacy-preserving robot teleoperation APIs.

This paper is organized as follows: Section 2 reviews related work. Section 3 describes the experimental setup and dataset collection. Section 4 characterizes robot teleoperation traffic patterns. Section 5 evaluates traffic fingerprinting attacks. Section 6 presents the information-theoretic privacy analysis. Section 7 discusses defenses, Section 8 addresses generalizability considerations, and Section 9 concludes the paper.

The present paper extends our previous conference work (Tang et al., 2025c), which focused on empirically demonstrating the feasibility of inferring robot actions from encrypted teleoperation traffic. In addition to the empirical analysis presented in that work, this paper applies and adapts an information-theoretic framework to quantify the intrinsic leakage of the protocol and derive Bayes optimal bounds on inference accuracy, as detailed in Sections 2.3 and 6. The manuscript has been correspondingly revised and expanded to integrate these additions and improve the overall presentation.

2. Related work

2.1. Analysis of encrypted network traffic

In communications established across the network, encryption protocols (e.g., TLS) ensure that data remains confidential and secure against unauthorized tampering. Despite the use of encryption, it is still possible to infer privacy-sensitive information pertaining to these data exchanges through the analysis of the *communication metadata*; characteristics such as the size, timing, and frequency of network packets comprising encrypted communications reveal patterns that allow external observers to infer communications' contents (Nguyen and Armitage, 2008).

Traffic *fingerprinting* is a sub-category of network traffic analysis. In a fingerprinting attack, an adversary's goal is to re-identify the occurrence of a certain event in the network, based on the similarity to past events labeled by the adversary, i.e., a set of traffic "fingerprints" (Hayes and Danezis, 2016). These attacks have been widely researched in the context of website fingerprinting (Xue et al., 2022; Rahman et al., 2020), where an adversary wishes to uncover which website a client is accessing through an encrypted tunnel (e.g., Virtual Private Networks (VPNs) Xue et al., 2022 or Tor Rahman et al., 2020) that hides the website's IP address from a local network observer.

2.2. Fingerprinting encrypted robotic teleoperation traffic

Teleoperated robots are controlled remotely via network commands, facing privacy challenges that bear a notable resemblance to those encountered in website fingerprinting. Specifically, traffic fingerprinting techniques in robotic teleoperation can expose fine-grained aspects of robot operations, ranging from movement patterns to user interactions, thereby posing a considerable privacy risk to users. Shah et al. (2022) demonstrated that robot movements can be identified from encrypted traffic using deep neural network-based analysis. Adversaries can extract detailed operational sequences of robots, potentially revealing sensitive information, or unauthorized replication of tasks. In earlier work (Tang et al., 2025b), we classified robot actions using higher-level APIs like Cartesian commands, achieving high accuracy through signal processing and traffic statistics analysis. However, these studies did not explore classifying daily actions and tasks using joint-level controllers, the most fundamental type of robotic arm controllers, thus leaving a critical gap in existing research.

As it stands, not all tasks can use high-level descriptions, especially in dynamic environments like homes or care facilities where human–robot interaction may occur. Unlike robots adopted in industrial applications, robots in these settings must adjust in real-time, needing to target delicate handling, human contact, or complex environments. For example, in rehabilitation, a robot assisting with arm lifting needs to continuously adapt its torque and position based on the



Fig. 1. The smartphone sends signals to the robot control software on a computer via cable, and the computer communicates with the robot over the same network.

patient's movements, requiring low-level control for smooth and safe interaction. In these cases, high-level controller commands need to be translated into precise joint-level position/velocity/torque commands that call for real-time sensor feedback. While Shah et al. (2022) inferred low-level motions from encrypted traffic, they did not explore the inference of high-level actions through the composition/analysis of low-level motions, raising the question of whether and to which degree such relationships can be established. In our work, we show that it is possible to predict high-level actions from encrypted network traffic using joint-level controllers.

2.3. Information-theoretic privacy bounding

In the domain of encrypted traffic analysis, privacy risks are traditionally evaluated empirically by training machine learning classifiers and reporting their accuracy. However, relying solely on empirical accuracy is fundamentally limited. A low classification accuracy may stem from a genuinely secure protocol, but it could equally result from a suboptimal classifier constrained by architectural limitations, insufficient hyperparameter tuning, or finite-sample optimization noise. Consequently, empirical evaluations systematically risk underestimating the true underlying privacy leakage.

To address this empirical–theoretical gap, researchers have introduced classifier-agnostic, information-theoretic frameworks to establish rigorous bounds on privacy. Notably, Li et al. (2018) proposed the Weighted Feature Dependence Estimator (WeFDE). Instead of training a discrete classifier, WeFDE leverages adaptive Kernel Density Estimation (KDE) and feature clustering to model the continuous probability distributions of high-dimensional traffic features. By computing the Shannon mutual information between these features and the target sensitive variables, WeFDE provides a deterministic measure of information leakage. This mutual information can be mapped through Fano's Inequality (Fano, 1961) to establish a Bayes optimal accuracy ceiling, representing the absolute maximum success rate achievable by any optimal adversary.

While information-theoretic bounding has been adopted to rigorously evaluate website fingerprinting defenses (Li et al., 2018), its application has remained strictly confined to web traffic analysis. To the best of our knowledge, this paper is the first to adapt this methodology to the continuous, physical domain of robotic teleoperation. In our work, we utilize a calibrated WeFDE methodology to definitively bound the structural privacy leakage across different joint-level controllers, isolating objective privacy risks independent of machine learning inefficiencies.

3. Experimental setup

3.1. Hardware and software configurations

Our experimental setup leverages a smartphone equipped with an Inertial Measurement Unit (IMU) consisting of an accelerometer, a gyroscope, and a magnetometer, which we use to teleoperate a commercial collaborative robot arm (Sawyer from Rethink Robotics) as depicted in Fig. 1.

Controllers used in teleoperation. We implemented teleoperation using the three types of joint-level controllers available on the Sawyer: position, velocity, and torque control, commonly used in commercial robot arms to move robot arms, where position and velocity control are desired for precision tasks and torque control for tasks requiring interactions with humans and/or delicate objects. The IMU data are collected from the smartphone to estimate its position and velocity, then, depending on the desired controller, translated into the robot arms' joint position, velocity, and torques for the arm to move with the smartphone. Ultimately, we used a single phone to collect network traffic generated via teleoperation (see Section 3.3). Our goal is to capture valid trajectories rather than precise motion measurements. Minor sensor differences do not affect overall motion, ensuring coherent trajectories for analysis.

Teleoperation software and secure communication. The control of Sawyer is facilitated through API[1] that enables the generation of command patterns for effective robot operation. To establish an encrypted communication channel between the robot and the computer, we used the built-in OpenVPN features of a commercial-off-the-shelf router. Commands are processed on this computer, running Ubuntu 20.04 LTS, which communicates with the robot via ROS (Robotic Operating System) Noetic, a standard framework in robotics. The robotic arm and computer are connected to the same router via Ethernet, ensuring reliable data transmission and real-time command execution/feedback.

3.2. Threat model

Our threat model assumes a network adversary who is able to intercept the encrypted network traffic flowing between the computer and the robotic arm (e.g., by wiretapping a physical Ethernet link). The adversary aims at identifying the (potentially privacy-sensitive) actions being performed by the robot, but is unable to break the cryptographic assumptions that would allow it to reveal the actual content of the encrypted communication channel.

To uncover the actions performed by the robot, the adversary will resort to encrypted traffic analysis techniques based on the inspection of the communication metadata exchanged between the controller and the robot, e.g., the timing, command frequency, and the size of network packets exchanged in the communication, as well as the overall volume of exchanged data. Similar to the website fingerprinting literature, we assume that the adversary can replicate the operational setup of their target — in this case, including network conditions, the use of the same robot models, and the use of the same encryption techniques to establish a secure communication channel between the controller and the robot.

In particular, the adversary will attempt to re-identify the occurrence of a set of actions by the target robot as follows: First, the adversary generates a comprehensive database of network traces ("fingerprints") by executing a set of actions that closely resemble the ones that may be executed by the target robot in the future. Then, the adversary waits for the target robot to perform an action whose information is exchanged via the encrypted tunnel, collects the robot's traffic, and attempts to match it with one of the fingerprints in its

[1] https://rethinkrobotics.github.io/intera_sdk_docs/5.1.0/index.html.

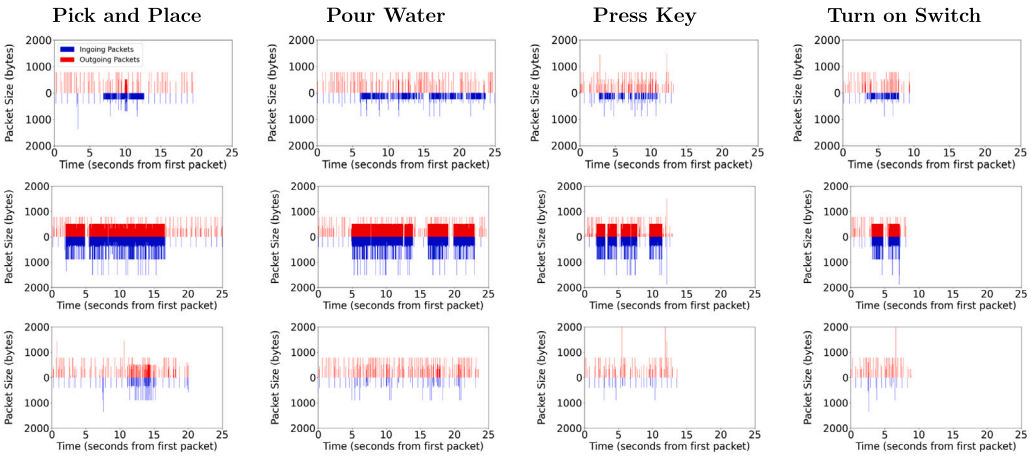


Fig. 2. Examples of four actions' packet lengths over time. (First row — position controller; second row — velocity controller; third row — torque controller.)

database. This matching is performed via machine learning (Hayes and Danezis, 2016) and deep learning (Sirinam et al., 2018; Rahman et al., 2020) techniques, which help automate and enhance the fingerprinting's effectiveness.

Traffic fingerprinting typically focuses on two different scenarios (referred to as *closed*- and/or *open-world* Sirinam et al., 2018), depending on the information made available to the adversary. In this work, we assume a *closed-world* scenario, where an adversary is given the knowledge about a limited set of actions a robot is able to perform (also known as the *monitored* set). While the set of monitored actions is limited, it may be sufficient to allow an adversary to infer a robot's activity in specific contexts. For instance, one may assume that a caregiving robot would perform a few pre-determined actions for helping a patient with reduced mobility, but that a different set of actions would be more relevant for a visually impaired patient. Should the adversary know which condition afflicts a patient, they could reasonably create a database of actions expected to be performed by the robot.

3.3. Dataset collection

Choice of robotic actions. We collected network traces (Tang et al., 2025a) while the robot performed *four* distinct actions: (a) picking up and placing an object; (b) pouring water; (c) pressing a key, and; (d) turning on a switch. This set of actions was selected based on the assumption that the robot may be operating in a private setting, where the four selected actions are highly representative of recurring daily routine actions. While these actions may seem basic, they may reveal user habits, leading to the exposure of private information and user profiling.

Among the four actions, *pour water* is mostly similar to *pick and place*, while *press key* is mostly similar to *turn on switch*. We deliberately chose two pairs of actions that could be considered dissimilar to one another, while internally similar within each pair. This approach allows for exploring how subtle differences in the network traffic generated by joint-level commands can help distinguish between similar actions, as well as demonstrating clear distinctions between different types of movements. This selection helps us evaluate the effectiveness of fingerprinting across varied, yet closely related, robot behaviors (Section 5.2).

Robotic actions' trajectories. To ensure our traces provide a robust basis for assessing fingerprinting performance, we collected each action sample following a *different trajectory*. In essence, action trajectories are generated via the execution of several sub-tasks (e.g., move to a given location, press/lift an object, etc.) that are chained together and executed at different instants in time. Each sub-task may be configured to operate at different robot movement speeds, to assume disparate movement waypoints and object locations, and other task-specific parameters (e.g., pouring angles). In addition, each trajectory may involve additional variability due to minor variations in sensor readings (see Section 3.1).

For each action class, we collected and interpolated 100 sample motion trajectories using the phone, each lasting 10 to 30 s, totaling 400 data samples. One traffic sample encompasses the entire network data exchanged between the robot and the controller PC when completing an instance of the desired action.

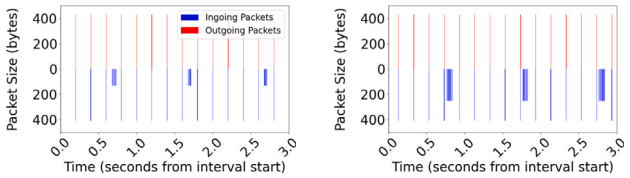
Per-controller data acquisition. We used the interpolated motion samples to operate the robot using our three different controllers: position, velocity, and torque. During these operations, we captured the network traffic exchanged between the robot and the computer. This process resulted in 400 network traffic trace samples for each type of controller, amounting to a total of 1200 network traffic trace samples. We acquired the network traces by running the Linux `tcpdump` utility on the robot's controller computer, which allows us to record the encrypted data exchanges between the robot and the controller.

4. Characterization of robot actions traffic

In our testbed, the command and feedback messages necessary for teleoperation are encrypted by VPN software, preventing adversaries from reading their content. However, different actions might produce distinct network patterns. Commands requiring more parameters may generate larger packets; different actions may require different command sequences, resulting in varying packet sizes; and different actions might require command and feedback exchanges at different frequencies, affecting packet inter-arrival times. These possibly disparate traffic patterns could allow an adversary to match them to specific actions.

To better visualize the existence of such patterns in robot teleoperation traffic, Fig. 2 presents a set of example network traces collected during the execution of different actions. (The traces comprising each action/controller combination were found to be rather stable.) Each row of the figure depicts the packet sizes and timing relationship for the position, velocity, and torque controllers. In this figure, "incoming" refers to the packets sent by the controller and received by the robot, while "outgoing" refers to packets sent in the reverse direction.

Different controller/action combinations produce disparate traffic patterns. A closer analysis of Fig. 2 allows us to pinpoint two major observations: (a) row-wise, each controller originates a different traffic pattern across each of the actions, and; (b) column-wise, for the same action, each controller originates a different network traffic pattern. Overall, this means that each action performed by each different controller will exhibit a different traffic pattern in the network,



(a) Command one joint. (b) Command all joints.

Fig. 3. Packet lengths over time when commanding a different number of joints.

potentially allowing traffic fingerprinting techniques to identify the traces produced by a specific controller/action combination.

Consider, for instance, the first row of Fig. 2, which showcases the traffic patterns produced by the position controller over each of the four actions. For each action, we can see that not only is each action performed in a different amount of time, but also that the frequency of packets exchanged between the robot and its controller is quite disparate. For instance, for the *pick and place* action, there is a high concentration of incoming packets from around 7 s to 12 s in the trace, while incoming packets for the *pour water* action form four individual clusters after 5 s into the trace, where these packets are more concentrated. The traffic patterns generated by other controllers given the same action follow a similar trend, i.e., the length of the packet exchange, as well as the frequency of packets exchanged at different points in the trace is indicative of the action being performed. Now, consider also the first column of Fig. 2, showcasing the traffic patterns for the *pick and place* action across three different controllers. We can observe that each controller will also produce a different pattern.

Packet sizes reflect the number of joints being commanded. Fig. 3 depicts a comparative analysis of command transmissions when controlling a single joint versus all seven joints of the robot. Note that sending one command to a given joint essentially requires the controller to specify a value for one key in a dictionary. Thus, the more joints are controlled simultaneously, the more information needs to be conveyed by the controller to the robot. Indeed, we can observe from the figure that packet lengths experience a noticeable increase when all seven joints are being controlled. This trend is also consistent across position, velocity, and torque control modes. Interestingly, this means that network traffic analysis techniques can also focus on these particularities to infer whether simpler/more complex actions are being performed, depending on the number of joints used by the robot at any given time. For instance, in a simple horizontal motion of the arm, only one joint needs to be moved. In this case, the controller would send smaller packets for commanding the movement of a single joint.

5. Evaluation of fingerprinting attacks

5.1. Traffic analysis attacks and metrics

Classical ML-based attacks. These attacks work by analyzing a set of manually-engineered features extracted from encrypted network traffic. These features include summary statistics such as the average packet size and percentiles of inter-packet timing and are used to identify patterns that can distinguish different types of traffic despite encryption. We use three ML-based classifiers (initially proposed for website fingerprinting) that operate on summary statistics, including k-Fingerprinting (Hayes and Danezis, 2016), CUMUL (Panchenko and Lanze, 2016), and k-Nearest Neighbors (kNN) (Wang et al., 2014). We also devised our own classifier based on gradient-boosted decision trees (XGBoost), resorting to summary statistics focusing the frequency distribution of packet sizes and timing, as well as each action's communication volume.

Deep Learning (DL) attacks. Unlike classical ML attacks which require manual feature engineering, deep learning-based attacks automatically extract features from network traces by using kernels as filters during the backpropagation process. These methods are generally more resource-intensive, requiring significant computational power and large training datasets. While acquiring such datasets is relatively straightforward in domains like website fingerprinting, it proves challenging in the context of robotic operations, where collecting large amounts of real-world data requires extensive work (O'Neill et al., 2024). Nevertheless, we explore these classifiers within our study, considering their success in network traffic classification.

We use two deep learning attacks based on Convolutional Neural Networks (CNNs), named deep fingerprinting (DF) (Sirinam et al., 2018) and Tik-Tok (Rahman et al., 2020). While both classifiers share the same neural network architecture, they use different input representations of the network data they are trained/tested on. Specifically, DF leverages an input vector formed by the *direction and size* of traffic (e.g., [+50, −50, +80] denotes a trace formed by an outgoing packet of size 50, an incoming packet of size 50, and an outgoing packet of size 80). Instead, Tik-Tok considers a *directional timing* input representation (e.g., [0, −0.02, +0.34] denotes a trace formed by the first outgoing packet that is sent to the network, followed by an incoming packet that arrives 0.02 s after the first packet was sent, and so on).

In addition to the "vanilla" inputs for both classifiers, we generated two alternative input representations. Specifically, we started by generating an alternative input representation for Tik-Tok, where we simply use raw packet timings (e.g., [0, 0.02, 0.54] denotes a trace where the second packet exchanged in the connection was sent after 0.02 s of the start of the connection). Then, for assessing the potential benefits of including all three of the metadata dimensions we consider for a network packet (direction, timing, and size), we crafted new input vectors containing these dimensions. We feed this hybrid feature vector into the generic DF/Tik-Tok architecture for building a new classifier we refer to as DF-Tok.

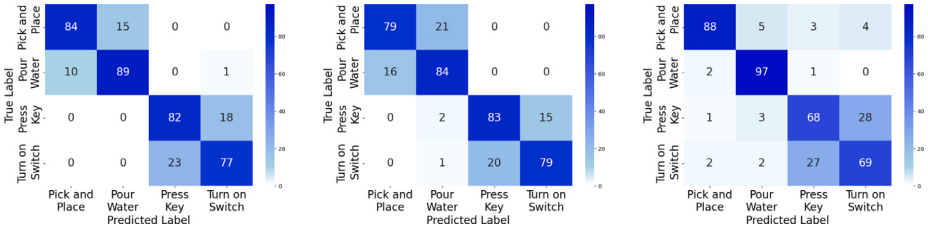
Lastly, we use another recent attack known as Robust Fingerprinting (RF) (Shen et al., 2023). This attack aggregates the direction and timing information of packets (according to fixed-length time intervals) into a matrix and uses a CNN-based classifier.

Classification metrics and setup. We evaluate the success of each attack through its *accuracy* and use *confusion matrices* to examine predictions across different classes. We used 10-fold cross-validation when experimenting with classical ML classifiers and, due to computational demands, used an 80% training/10% validation/10% testing splits when evaluating our deep learning classifiers. All results refer to the accuracy obtained on each classifier's testing set.

5.2. Attack results

Table 1 depicts the accuracy of each attack when identifying the action performed by the robot under a given controller. These results showcase significant privacy risks (accuracy surpassing 80%), suggesting that an adversary could potentially identify the actions executed by the robot by using already existing fingerprinting attacks. Below, we describe our main findings.

XGBoost performs best in uncovering robot actions from encrypted network traffic. Fig. 4 displays the confusion matrices for the three controllers using the XGBoost classifier, which yielded the highest accuracy. The actions of picking and placing an object and pouring water tend to overlap in terms of their characteristic patterns, which can lead to these actions being more easily confused with one another during classification. Similarly, the actions of pressing a key and the tapping motion involved turning on a switch and exhibit a higher



(a) Position controller. (b) Velocity controller. (c) Torque controller.

Fig. 4. XGBoost's confusion matrices for the three controllers.

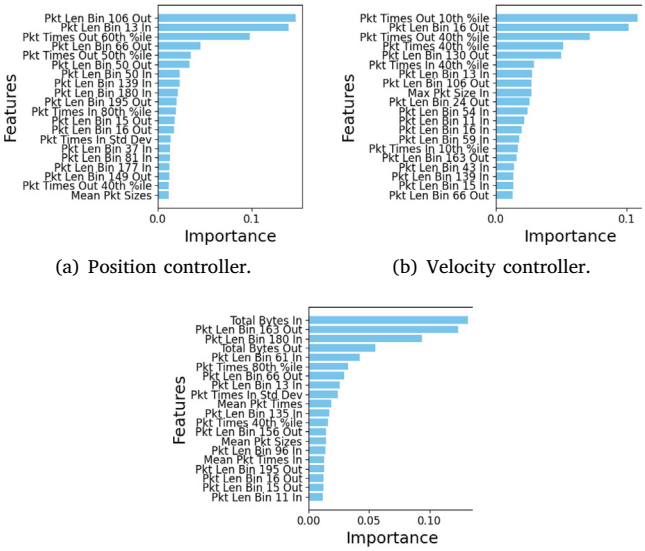
Table 1
Accuracy of each attack in distinguishing between the four different actions across each of the controllers.

Attack	Position (%)	Velocity (%)	Torque (%)
k-Fingerprinting	77.2	63.5	72.5
KNN	33.2	31.0	27.5
CUMUL	63.6	61.0	63.8
XGBoost	83.2	81.2	80.5
DF (direction, sizes)	67.5	62.5	48.8
Tik-Tok (direction, timing)	28.8	27.5	28.8
Tik-Tok (timing)	28.8	27.5	28.8
DF-Tok (direction, timing, sizes)	67.5	63.2	55.0
RF (direction, timing)	38.0	37.8	37.4

likelihood of being mixed up. This observation aligns with the intuitive understanding that actions sharing similar physical movements or operational characteristics are more challenging to distinguish based on traffic patterns.

The performance of deep learning attacks was subpar when compared to those based on classical ML. Despite the theoretical advantages of CNNs, the performance of the deep learning attacks we considered may have been limited by several factors. First, these attacks were proposed for the website fingerprinting setting, where data collection is a rather easy process to automate and conduct at scale, whereas this is not the particular case for teleoperation. Second, while Tik-Tok and RF take packet timing into consideration, they forego packet sizing information, thus being unable to build important context about the trace by distinguishing between different packet types composing a trace (and which may only be distinguishable according to their size). This may present a challenge to these classifiers, since, alongside the command messages we rely on to characterize each action, the controller and the robot exchange maintenance and feedback messages that are not useful for action identification. Indeed, DF-Tok's results suggest that timing information appears to provide only limited gains on top of DF with packet direction and size information. However, performance gains were evident when applied to the torque controller (~6% increase). In contrast, attacks that jointly consider statistics about both packet sizes and timing have demonstrated a compelling potential to unveil sensitive data. Classical ML attacks, including k-Fingerprinting or our XGBoost-based classifier, were effective in uncovering the actions executed by the teleoperated robot.

Beyond dataset size constraints, the nature of robot teleoperation traffic may also help explain why CNN-based classifiers underperform classical ML approaches in this setting. On the one hand, classical ML-based classifiers typically operate on hand-engineered summary statistics, such as percentiles of packet sizes, inter-packet timings, and total communication volume, which form a low-dimensional tabular feature space. Gradient-boosted decision trees (as in XGBoost) can exploit this structure efficiently via splits that capture, for instance,



(a) Position controller. (b) Velocity controller.

(c) Torque controller.

Fig. 5. Top 20 feature importance for three controllers using XGBoost.

the discrete distributions of packet length bins. On the other hand, CNN-based classifiers such as DF and Tik-Tok treat each trace as a sequential signal and rely on convolutional filters to detect local patterns. While this approach is well-suited to the website fingerprinting domain, where traces are long and encode intricate browsing dynamics, robot teleoperation traces are short and dominated by a small number of recurrent packet types of predictable sizes. As suggested by our feature importance analysis (see Fig. 5, described on the next bullet), the primary discriminative signals in the robotic teleoperation domain are global distributional statistics, which classical ML classifiers are well positioned to exploit.

Different controllers produce different sets of traffic characteristics useful for action identification. Fig. 5 showcases the top 20 most important features for classifying actions across the three different controllers, as determined by our XGBoost classifier. It can be observed that the position, velocity, and torque controllers are all influenced by the number of packets exchanged in a trace, featuring a mix of useful packet size ranges that are both shared and distinct across controllers. This is also true for some indicators of timing information between packets. Additionally, the total in/out bytes transmitted in the connection are relevant for the torque controller, but not for the other controllers. This is consistent with the third row of Fig. 2, where it can be seen that different actions tend to produce traffic sequences of varying length (e.g., the tap action is relatively short, under 10 s,

whereas the pour water action is longer, over 20 s), suggesting that longer action sequences also generate a larger amount of packets, providing useful information to the classifier.

Some actions share common feature distributions. When delving into the per-class distribution of the most important feature for each controller, we identified an overlap in the feature distributions for some classes, revealing some of the inherent difficulties in distinguishing between the network traces of different actions. For instance, consider the *10th percentile of outgoing packets' times* feature, identified as the most important feature for distinguishing actions performed with the velocity controller (Fig. 5(b)). We observed a clear similarity between the distribution of this feature among the samples of the *pick and place* and *pour water* actions (both ranging from 4 to 6 ms), and also between *press key* and *turn on switch* (2 to 4 ms), whereas there is an observable difference between these two groups of actions. This is further evidenced by the results shown in the confusion matrix of Fig. 4(b).

6. Information-theoretic privacy analysis

While our empirical ML and DL results in Section 5 demonstrate the feasibility of action inference from encrypted teleoperation traffic, they do not reveal whether the remaining classification errors arise from limitations of the models or from intrinsic properties of the communication protocol. To address this ambiguity, we adapt and apply a classifier-agnostic information-theoretic evaluation that estimates the fundamental privacy leakage of the system.

6.1. Information-theoretic bounding framework

We estimate the mutual information between the observable network traffic features and the robot actions being executed. This allows us to quantify the amount of information about the robot's behavior that is inherently leaked through the encrypted communication channel.

By estimating the joint Shannon mutual information $I(X;Y)$ between the continuous high-dimensional traffic features X and the discrete robotic actions Y, and mapping this leakage through Fano's Inequality in Section 6.2, we derive an upper bound on the theoretical classification accuracy achievable by any adversary. This bound is independent of empirical model limitations such as classifier design, heuristic optimization, or finite-sample effects. Our analysis evaluates mutual information over a comprehensive set of traffic features capturing the spatial and temporal characteristics used across our empirical classifiers. In contrast, practical machine learning models operate within restricted hypothesis spaces and may fail to exploit all informative feature interactions. The resulting gap between empirical accuracy and the theoretical bound therefore reflects deterministic information present in the traffic that empirical models do not fully capture. This gap can be further amplified by the "curse of dimensionality", which makes learning reliable decision boundaries increasingly difficult in high-dimensional feature spaces.

To translate this theoretical bound into a concrete accuracy limit, we map the estimated mutual information (in bits) to a classification accuracy ceiling. Let X denote the high-dimensional traffic features and $Y \in \mathcal{Y}$ the discrete robotic actions, where the total number of classes is $|\mathcal{Y}| = 4$. Assuming a uniform prior distribution of actions in our teleoperation scenario, the initial uncertainty is maximized at the prior entropy $H(Y) = \log_2(|\mathcal{Y}|) = 2.0$ bits.

Once the joint mutual information $I(X;Y)$ is estimated, it quantifies how much information about the robotic actions is revealed by the observable traffic features. According to information theory, the remaining uncertainty about the actions, given the observed traffic, is captured by the conditional entropy:

$$H(Y|X) = H(Y) - I(X;Y). \tag{1}$$

Fano's inequality provides a lower bound on the probability of classification error $P_e = \Pr(\hat{Y} \neq Y)$, for any predictive estimator $\hat{Y}$ derived from X:

$$H(Y|X) \leq H_b(P_e) + P_e \log_2(|\mathcal{Y}| - 1) \tag{2}$$

where $H_b(P_e) = -P_e \log_2(P_e) - (1 - P_e)\log_2(1 - P_e)$ is the binary entropy function.

By substituting the empirically estimated conditional entropy into the inequality, we establish a strict boundary condition. Because the right-hand side of the inequality is monotonically increasing with respect to P_e (for $P_e \leq 1 - 1/|\mathcal{Y}|$), we can solve the equality numerically to determine the minimum achievable Bayes error rate, denoted as P_e^*. The absolute theoretical maximum accuracy achievable by any optimal adversary is then strictly bounded by $1 - P_e^*$.

6.2. Bounding and estimator configuration

We use a methodology inspired in WeFDE (Li et al., 2018) to approximate this joint mutual information. WeFDE employs an additive clustering heuristic: it groups dependent features into clusters, calculates the joint Kernel Density Estimation (KDE) for each cluster, and sums their mutual information. However, in low-entropy domains like our 4-class teleoperation scenario (maximum entropy $H(Y) = 2.0$ bits), residual correlations cause this heuristic to double-count redundant information, inflating the leakage until it saturates the 2.0-bit mathematical maximum.

To extract a rigorous bound and mitigate these algorithmic artifacts, we calibrated the estimator's hyperparameters. First, to preclude cluster double-counting, we discarded default settings and enforced a Normalized Mutual Information (NMI) redundancy threshold of 0.4 ($\tau = 0.4$). This acts as an orthogonality filter, ensuring that the retained top-K features encode distinct dimensions (e.g., macroscopic volume vs. microscopic timing). Second, to ensure convergence of the KDE integrals, we amplified the Monte Carlo sampling resolution to $N = 10{,}000$ synthetic instances.

To evaluate the maximum information leakage captured by the joint feature set, we must progressively incorporate features into the analysis. The original application of WeFDE in website fingerprinting evaluates large feature subsets (e.g., $K \geq 50$) to capture information across high-entropy domains involving hundreds of classes (e.g., $H(Y) \geq 6.6$ bits). In contrast, our robot teleoperation scenario is a constrained domain with a maximum entropy of $H(Y) = 2.0$ bits (four actions). Because a single dominant feature captures a substantial fraction of this capacity, incorporating even a small number of features rapidly saturates the true mutual information. Evaluating a large dimensionality cut-off would trigger the estimator's double-counting artifact, inflating the estimation beyond the mathematical maximum.

Consequently, we evaluate feature inclusion starting from a single feature ($K = 1$) in discrete integer steps ($K \in \mathbb{Z}^+$). Rather than applying an arbitrary dimensionality cut-off, we dynamically bound the feature retention (K) via marginal information gain (ΔI) analysis. According to the *Submodularity of Information*, physical leakage (i.e., the intrinsic information about robot actions that is encoded in the teleoperation traffic generated by the underlying control process) must exhibit diminishing marginal returns ($\Delta I_{K+1} \leq \Delta I_K$) prior to the onset of additive artifacts. By tracking ΔI incrementally from $K = 1$, we isolate the plateaus corresponding to the intrinsic physical leakage before the estimator's clustering logic breaks down. This bounds the continuous supremum of true mutual information prior to artifact onset.

To validate that our reported bounds represent physical realities rather than artifacts of estimator tuning, we conducted a hyperparameter sensitivity analysis. We evaluated the joint mutual information across extreme thresholds: the permissive default ($\tau = 0.9$) and our orthogonality constraint ($\tau = 0.4$). Modulating τ dictates the composition of the selected feature subset. While $\tau = 0.9$ permissively ingests correlated feature variants (e.g., redundant statistical derivations of the

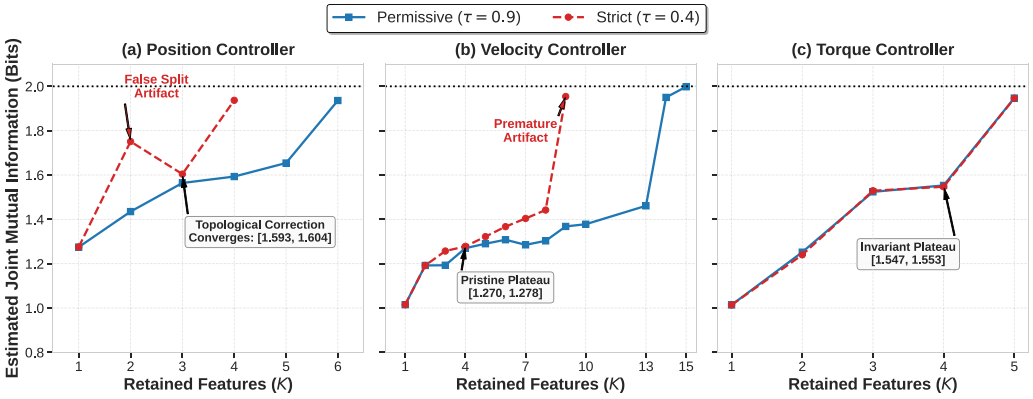


Fig. 6. Estimated joint mutual information as additional traffic features are incorporated (K = number of retained features), under a permissive redundancy threshold ($\tau = 0.9$, blue solid) and a strict orthogonality threshold ($\tau = 0.4$, red dashed). A genuine plateau indicates that all physically meaningful signals have been captured; rises beyond the plateau reflect double-counting artifacts in the estimator's clustering logic. We identify the plateau by tracking marginal information gains (ΔI) and stopping when submodularity is violated ($\Delta I_{K+1} > \Delta I_K$). A plateau is labeled as *Pristine* when identified under the permissive threshold before artifact onset (e.g., Fig. 6(b)), and as *Invariant* when stable across both threshold configurations (e.g., Fig. 6(c)). The remaining annotations mark where the estimator begins inflating its estimate due to clustering instabilities (e.g., *False Split Artifact*, *Premature Artifact*) or where the introduction of an additional feature corrects a prior overestimate by forcing features back into a single cluster (*Topological Correction*). The convergence of both threshold configurations ($\tau = 0.9$ and $\tau = 0.4$) to the same plateau range confirm that the reported bound reflects the protocol's structural information. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

dominant physical signal), the $\tau = 0.4$ filter purges these redundancies, forcing the estimator to mine deeper into the feature rankings to select strictly orthogonal dimensions (i.e., statistically independent informational channels). Consequently, the K-dimensional subsets evaluated under the two thresholds consist of mathematically distinct components of the network traffic.

6.3. Estimator robustness and artifact correction

As shown for the *Velocity* controller in Fig. 6(b), under the permissive $\tau = 0.9$ threshold, the estimator ingested correlated feature variants. Consequently, the HDBSCAN clustering step within the estimator grouped these redundant dimensions into a single cluster. This prevented the additive summation artifact, prolonging a plateau before encountering high-dimensional collapse at $K = 14$. We refer to this plateau as *Pristine*, as it is observable under the permissive threshold, prior to any artifact onset (see next paragraph). This is in contrast to the *Invariant Plateau* observed for the Torque controller (Fig. 6(c)), where the plateau is stable across both thresholds.

Conversely, enforcing the strict $\tau = 0.4$ orthogonality filter forced the estimator to purge redundant traits and select independent dimensions. This spatial sparsity partitioned the features into distinct clusters, accelerating the estimator's additive double-counting artifact and triggering saturation at $K = 9$ (1.953 bits), earlier than under the permissive threshold ($K = 14$) and without a subsequent self-correction. We refer to this as a *Premature Artifact*, to distinguish it from the *False Split Artifact* observed in the *Position* controller (Fig. 6(a)), where the estimate self-corrects the submodularity violation upon the addition of a bridging feature.

Despite these different saturation timelines and feature compositions, tracking the marginal information gain (ΔI) isolated the same physical plateau. Under $\tau = 0.4$, the estimation obeyed submodularity (diminishing marginal returns) up to $K = 4$, peaking at 1.278 bits ($\Delta I = +0.022$), before violating submodularity at $K = 5$ ($\Delta I = +0.043$) due to artifact onset. This convergence, where the physical plateau stabilizes within the $[1.270, 1.278]$ bits envelope under both configurations, supports the robustness of our empirical Fano ceiling (~87.8% for *Velocity*). It demonstrates that our reported bound is not an artifact of estimator tuning, but rather the structural information limit of the physical protocol.

Clustering instability and artifact correction in position control. Our hyperparameter sensitivity analysis on the *Position* controller (Fig. 6(a)) highlighted the necessity of tracking marginal information dynamics over reliance on raw estimator outputs. Under the orthogonality constraint ($\tau = 0.4$), the estimator exhibited a non-monotonic trajectory: peaking at $K = 2$ (1.751 bits) before dropping at $K = 3$ (1.604 bits). In information theory, mutual information cannot decrease when additional dimensions are included.

This violation unmasks a topological instability in the underlying HDBSCAN clustering engine. At $K = 2$, the spatial sparsity forced the algorithm to partition the features into discrete clusters, thereby triggering an early additive double-counting artifact (1.751 bits), which we refer to as *False Split Artifact*. However, the introduction of the third feature at $K = 3$ acted as a spatial bridge, rectifying the topological density and forcing the algorithm to re-merge the dimensions into a unified manifold (we refer to this event as a *Topological Correction*). This merging deactivated the additive summation flaw, returning the estimation to its joint KDE integration of 1.604 bits. This artifact-corrected joint evaluation at $\tau = 0.4$ ($K = 3$, 1.604 bits) converges with the submodular plateau observed under the $\tau = 0.9$ threshold ($K = 4$, 1.593 bits), maintaining a divergence of ~0.011 bits. This cross-threshold convergence bounds the information ceiling of the *Position* protocol to a theoretical envelope of $[1.593, 1.604]$ bits. Translating this ceiling bounds the Bayes optimal adversarial accuracy at ~94.5%.

Furthermore, this threshold reduction yielded minimal deviations in the joint mutual information plateaus (e.g., stabilizing within $[1.547, 1.553]$ bits for the *Torque* controller (Fig. 6(c)) under both thresholds). This hyperparameter invariance supports two points. First, it confirms the reproducibility of our theoretical ceiling, precluding algorithmic cherry-picking. Second, it demonstrates that the primary discriminative features governing the robotic traffic are orthogonal (possessing pairwise NMI < 0.4). They encode independent dimensions of the teleoperation control stream (e.g., macroscopic sequence volume versus microscopic packet timing) without requiring the estimator to penalize statistical overlaps.

Despite evaluating different multi-dimensional feature subsets, the marginal information trajectories consistently converged to the same

Table 2
Controllers' empirical performance vs. absolute information-theoretic bounds.

Controller	Empirical XGBoost Acc.	True joint MI bounding envelope	Theoretical Max Acc. (Fano envelope)	Optimality gap (Δ_{min})
Position	83.2%	[1.593, 1.604] bits	[94.3%, 94.5%]	~11.1%
Velocity	81.2%	[1.270, 1.278] bits	[87.7%, 87.9%]	~6.5%
Torque	80.5%	[1.547, 1.553] bits	[93.4%, 93.6%]	~12.9%

mutual information plateau. This convergence indicates that the reported ceilings are not artifacts of a particular feature selection, but rather reflect the intrinsic information leakage of the underlying teleoperation protocol. Consequently, these bounds represent the maximum inference capability available to any adversary observing the encrypted traffic, regardless of the specific feature engineering strategy employed.

6.4. Cross-controller physical insights and optimality gaps

Synthesizing the theoretical bounds across all three controllers unveils a counter-intuitive insight that empirical metrics alone obscure (summarized in Table 2).

A comparison between our empirical baselines and the theoretical ceilings highlights the impact of these algorithmic limitations across all control modalities. For example, using the *Position* controller as a baseline, our XGBoost model achieved an empirical accuracy of 83.2%. However, our bounded WeFDE analysis indicates that the underlying physical protocol leaks more information, corresponding through Fano's Inequality to a Bayes optimal accuracy limit of up to ~94.5%. This ~11.3% gap directly reflects the portion of exploitable signal that the empirical classifier fails to capture due to the distributed nature of the predictive features. While this fundamental fragmentation penalty applies globally, comparing how this optimality gap varies across the Velocity and Torque controllers reveals our most critical physical insights into how different control modalities structure the observable traffic leakage.

Without theoretical bounds, one might misattribute the empirical accuracy drop from the Position controller (83.2%) to the *Velocity* controller (81.2%) to sub-optimal ML hyperparameter tuning or insufficient feature engineering. However, applying our plateau-bounding WeFDE methodology reveals a structural property of the teleoperation protocol: the network traffic generated by the Velocity controller leaks less physical mutual information about the executed actions. By tracking the marginal information gain (ΔI), we isolated the plateau for the Velocity controller features within the envelope of $[1.270, 1.278]$ bits. Using Fano's Inequality, this leakage corresponds to a theoretical maximum classification accuracy at ~87.8%. Compared to the higher theoretical ceiling of the Position controller, this result suggests: (1) the Velocity protocol reduces the amount of deterministic signal at the physical layer, generating more overlapping network traffic distributions; (2) our XGBoost model's 81.2% accuracy leaves a ~6.5% gap to the deterministic limit of the Velocity protocol, indicating that the empirical ML model operates near the algorithmic limit.

Empirically, the *Torque* controller yielded the lowest XGBoost accuracy (80.5%), suggesting it generates the most secure traffic patterns. However, our WeFDE information-theoretic analysis challenges this assumption: the Torque controller leaks $[1.547, 1.553]$ bits of mutual information, translating to a Bayes optimal ceiling of ~93.6%. This results in a 12.9% empirical–theoretical gap compared to the Velocity controller. Unlike the Velocity protocol, which creates overlapping traffic patterns that inherently limit inference, the Torque protocol preserves distinguishable physical signals (e.g., macroscopic variations in total volume and sequence lengths, as noted in our feature importance analysis). However, Torque encodes these signals in complex, non-linear dependencies between features that step-function models like XGBoost struggle to exploit, leading to the observed 12.9% optimality gap. This theoretical insight aligns with our earlier Deep Learning findings: our hybrid DF-Tok CNN, which is better suited to capturing sequential traffic dependencies, exhibited performance gains (~6% increase) on the Torque controller. By utilizing WeFDE to estimate the theoretical information limits, we demonstrate that the additional deterministic information is physically present in Torque traffic, which requires more sophisticated architectural capacity to decode.

7. Towards effective fingerprinting defenses

These findings indicate that teleoperation protocols can inherently leak actionable information through their traffic patterns, motivating the need for privacy-aware communication designs. In designing software APIs for teleoperated robotic systems, several strategies can be considered to this end. As an inspiration, the literature on website fingerprinting is already prolific with the development of defenses to scramble traffic patterns so as to make fingerprinting attacks unsuccessful (Mathews et al., 2023). However, directly applying such techniques to robotic teleoperation is non-trivial due to the strict latency, reliability, and control-loop constraints imposed by cyber–physical systems. These constraints introduce unique design requirements for teleoperation traffic defenses.

7.1. Requirements for teleoperation traffic analysis defenses

Accommodating real-time constraints. Ideally, defense mechanisms should avoid compromising a robot's operational effectiveness. For instance, while strategies involving the padding of command packets to the same size and forcing packet delays might deter an adversary from identifying the robotic actions being performed, the bandwidth and latency overheads imposed by such strategies should not compromise the correct and timely operation of the robot. In contrast to web traffic, robot teleoperation is often ruled by real-time requirements which are not amenable to fingerprinting defenses that introduce latency. In general, if any data at the joint control level is delayed even by milliseconds, this would cause either the robot to stop (if such a feature is implemented on the onboard computer) or the controller could become unstable.

Zero-delay defenses. Defenses that avoid introducing delays have been recently explored in the website fingerprinting literature. FRONT (Gong and Wang, 2020) foregoes hefty bandwidth expenditure, altering traffic patterns by randomly introducing dummy packets during a connection's lifetime. However, FRONT does not pad a connection's "true" packets, potentially enabling fingerprinting attacks that leverage the existence of packet exchanges with predictable sizes/frequency as in the robotic traffic we characterized earlier in Section 4.

7.2. Prototyping a command size padding defense

We prototyped a simple simulator for a zero-delay defense against robot traffic fingerprinting, applying it on the traces we have previously collected (Section 3.3).

Since our earlier feature analysis (see Fig. 5) suggested that packet sizes are amongst the most important features for classification, we built a defense that artificially pads joint-level command packets to their nearest multiple of n bytes. We applied the defense in several configurations, including n = 200B, 400B, 800B, or 1800B. Given that the largest command packets we observed are in a size range

Table 3
XGBoost attack accuracy and bandwidth overhead (% of extra transmitted bytes) on padded command packets.

Padding (B)	Position		Velocity		Torque	
	Acc. (%)	OH (%)	Acc. (%)	OH (%)	Acc. (%)	OH (%)
200	80.7	121.1	72.9	132.9	79.7	120.7
400	79.7	136.1	72.5	164.5	79.3	137.1
800	78.5	182.2	73.4	252.2	78.7	182.6
1800	78.0	300.6	72.9	495.3	78.8	302.2

of 1600–1700B, the 1800B padding configuration essentially turns all command packets into the same size. Thus, this modification removes any useful information that could be extracted from the size of individual command packets alone. Padding increases bandwidth consumption (see Table 3), but does not affect packet transmission time nor adds extra packets to the link.

Standalone packet padding is ineffective. We assess the ability of the XGBoost classifier to distinguish actions protected by the defense by training and testing the classifier on the same (but now defended) subsets of the data used to train/test the original attacks in Section 5.2. The results of this experiment, shown in Table 3, reveal that padding command traffic alone appears to be largely ineffective for preventing the fingerprinting of position and torque controllers, even when padding all command packets to the same size, imposing a maximum ~8% accuracy degradation when classifying actions using the velocity controller. According to our feature importance analysis, this may be explained by the fact that, while the size of individual command messages is padded to the same length, the communication volume and amount of command packets exchanged in each direction of the communication still reveal sufficient information for the classifier to identify different actions.

Overall, our findings call for a comprehensive exploration of defense mechanisms that can withstand fingerprinting while ensuring the robot's operational effectiveness. While our evaluation of padding defenses is limited to a single strategy, future work may take inspiration from more advanced approaches which, in addition to packet padding, adaptively insert dummy traffic (Shmatikov and Wang, 2006) or regularize traffic bursts (Holland and Hopper, 2022) sent over a short period of time.

7.3. Towards effective fingerprinting defenses for robotic teleoperation

Mirroring defenses in website fingerprinting literature, the most effective strategy for robotic teleoperation would simultaneously obfuscate three key side channels: the total number of packets exchanged per action, the distribution of packets' inter-arrival times, and the distribution of packets' sizes. Indeed, padding and enforcing packet exchanges at a constant rate, similarly to what CS-BuFLO (Cai et al., 2014a) or Tamaraw (Cai et al., 2014b) accomplish for web traffic, could flatten packet count and timing distributions. This would decouple observable traffic volume from the underlying action duration. Moreover, sending a set of commands to all joints at a fixed rate, even in the absence of updates, would also mask the number of joints involved in motion execution (an information leak we underlined in Section 4).

However, constant-rate padding strategies are expected to incur high bandwidth costs, as maintaining a constant packet rate requires transmitting dummy data even when no meaningful updates are pending. Moreover, should the robotic application need to transmit more data than the allotted fixed rate allows, commands would need to be queued and delayed, which is particularly problematic given the strict real-time constraints of teleoperation. An alternative direction is to redesign teleoperation APIs to batch joint updates into fixed-size, fixed-rate network frames regardless of the number of joints commanded, removing the packet size and robot joint-count correlation identified in Section 4. We encourage future work to prototype and evaluate defenses along these axes, using accuracy bounds as derived in Section 6 as a baseline for measuring their effectiveness.

8. Considerations on generalizability

Robot characteristics. Different robots may use different joint-level control APIs and control modes, as well as different networking protocols to exchange data. This may result in different command formats and structure of associated messages/network packets. While we find that the Sawyer robotic arm is vulnerable to traffic fingerprinting (and likely, other ROS-based robots as well), we aim to conduct future work that extends our study to a wider set of robots.

Sets of actions. Our current evaluation considers four actions organized into two pairs of similar behaviors (*pick-and-place/pour water*; *press key/turn on switch*). While this experimental methodology exercises classifiers' ability to distinguish different (yet even relatively similar) robotic traffic patterns, it does not reflect the sheer diversity of tasks a home care robot may perform in practice. As the number of classes grows, the prior entropy $H(Y) = \log_2(|Y|)$ increases, raising the theoretical information ceiling while simultaneously placing greater demands on any fixed feature set. Under the WeFDE framework, a larger action space requires higher mutual information to sustain equivalent Bayes optimal bounds, and classifiers may empirically face greater inter-class traffic overlap. We recognize the task of extending the dataset to include a broader set of actions (including compound or long-horizon tasks) as an important direction for future work.

Open-world setting. Our threat model (Section 3.2) adopts a *closed-world* assumption, where the adversary knows the full set of actions the robot may perform (known as the monitored set) and attempts to re-identify which one is occurring. In practice, a robot may also execute actions outside the attacker's fingerprint database, and which may also include transitions between tasks or idle states. In an *open-world* setting, the classifier must additionally decide whether an observed trace belongs to any monitored class at all. Prior work in website fingerprinting has shown that the open-world setting can substantially reduce attack precision, as unmonitored traffic shares statistical properties with monitored classes, inflating false positive rates. We expect a similar degradation in our setting, and extending our evaluation to an open-world setting is left as future work.

Network conditions. Different network conditions may affect the traffic patterns observed on the communication between a robot and its controller. For instance, degraded conditions may cause packets to be dropped and retransmissions to occur, altering packet delivery rates or causing the exchange of additional messages. We aim to extend our study to more complex networking setups.

Information-theoretic analysis and WeFDE bounds reproducibility. The information-theoretic bounds we report depend on the traffic patterns observed in the collected dataset and the considered feature space. Different teleoperation protocols, communication stacks, or feature representations may alter the amount of information encoded in the observable traffic.

As detailed in Section 6.2, we take multiple configuration choices for governing our WeFDE-based accuracy bounding procedure. These include a normalized mutual information redundancy threshold of $\tau = 0.4$, a Monte Carlo sampling resolution of $N = 10{,}000$ synthetic instances for KDE convergence, and a dynamic feature retention cutoff determined by marginal information gain (ΔI) rather than a fixed K. We identified plateaus as the last K for which submodularity holds ($\Delta I_{K+1} \leq \Delta I_K$), and our hyperparameter sensitivity analysis confirmed that the reported plateaus are stable across different τ configurations.

Researchers applying our methodology to additional robotic platforms and teleoperation interfaces should replicate this sensitivity check; substantial divergence across threshold configurations would indicate either a higher-entropy domain requiring a larger K evaluation range, or a feature space with more complex correlation structure requiring additional artifact correction.

9. Conclusions

Our study uncovered significant privacy risks in robot teleoperation, particularly by analyzing encrypted network traffic data of joint-level controller actions exchanged between a collaborative robot arm and the computer that runs the controller. Our results show that robot actions can be inferred from encrypted traffic patterns, and our information-theoretic analysis further reveals the intrinsic amount of information about these actions that is structurally leaked by the teleoperation protocol. One possible step to address this issue is to integrate traffic scrambling methods in API design for robots, while balancing the integration of these privacy-preserving techniques with the efficiency of robot operations.

CRediT authorship contribution statement

Cheng Tang: Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Diogo Barradas:** Writing – review & editing, Resources, Funding acquisition, Conceptualization. **Urs Hengartner:** Writing – review & editing, Funding acquisition, Conceptualization. **Yue Hu:** Writing – review & editing, Supervision, Funding acquisition, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported in part by a seed grant jointly awarded by the University of Waterloo's and Cybersecurity Privacy Institute (CPI) and RoboHub. We thank RoboHub for providing us with access to the Sawyer robotic arm.

Data availability

Data will be made available on request.

References

Cabrera, M.E., Bhattacharjee, T., Dey, K., Cakmak, M., 2021. An exploration of accessible remote tele-operation for assistive mobile manipulators in the home. In: IEEE international conference on robot and human interactive communication (RO-MAN).

Cai, X., Nithyanand, R., Johnson, R., 2014a. Cs-buflo: A congestion sensitive website fingerprinting defense. In: Proceedings of the 13th Workshop on Privacy in the Electronic Society.

Cai, X., Nithyanand, R., Wang, T., Johnson, R., Goldberg, I., 2014b. A systematic approach to developing and evaluating website fingerprinting defenses. In: Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security.

DeMarinis, N., Tellex, S., Kemerlis, V.P., Konidaris, G., Fonseca, R., 2019. Scanning the internet for ROS: A view of security in robotics research. In: IEEE-RAS International COnference on Robotics and Automation (ICRA).

Fano, R.M., 1961. Transmission of Information: A Statistical Theory of Communications. MIT Press.

Gong, J., Wang, T., 2020. Zero-delay lightweight defenses against website fingerprinting. In: Proc. of USENIX Security Symposium.

González, C., Solanes, J.E., Munoz, A., Gracia, L., Girbés-Juan, V., Tornero, J., 2021. Advanced teleoperation and control system for industrial robots based on augmented virtuality and haptic feedback. J. Manuf. Syst. 59.

Hayes, J., Danezis, G., 2016. K-fingerprinting: A robust scalable website fingerprinting technique. In: Proc. of USENIX Security Symposium.

Holland, J.K., Hopper, N., 2022. RegulaTor: A straightforward website fingerprinting defense. Proc. Priv. Enhancing Technol..

Li, S., Guo, H., Hopper, N., 2018. Measuring information leakage in website fingerprinting attacks and defenses. In: Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security. CCS, pp. 1977–1992.

Mathews, N., Holland, J.K., Oh, S.E., Rahman, M.S., Hopper, N., Wright, M., 2023. SoK: A critical evaluation of efficient website fingerprinting defenses. In: Proc. of IEEE Security & Privacy (S&P).

Nguyen, T.T., Armitage, G., 2008. A survey of techniques for internet traffic classification using machine learning. IEEE Commun. Surv. & Tutor. 10 (4).

O'Neill, A., Rehman, A., Maddukuri, A., Gupta, A., Padalkar, A., Lee, A., Pooley, A., Gupta, A., Mandlekar, A., Jain, A., et al., 2024. Open x-embodiment: Robotic learning datasets and rt-x models: Open x-embodiment collaboration 0. In: IEEE International Conference on Robotics and Automation (ICRA). IEEE, pp. 6892–6903.

Panchenko, A., Lanze, F., 2016. Website fingerprinting at internet scale. In: Proc. of NDSS.

Rahman, M.S., Sirinam, P., Mathews, N., Gangadhara, K.G., Wright, M., 2020. Tik-tok: The utility of packet timing in website fingerprinting attacks. Proc. Priv. Enhancing Technol. 2020 (3).

Rodriguez, D., Perez, C., Jagersand, M., Figueroa, P., 2017. A comparison of smartphone interfaces for teleoperation of robot arms. In: Proc. of Latin American Computer Conference (CLEI).

Romeo, L., Petitti, A., Marani, R., Milella, A., 2020. Internet of robotic things in smart domains: Applications and challenges. Sensors 20 (12).

Sankar, G., Djamasbi, S., Li, Z., Xiao, J., Buchler, N., 2023. Systematic literature review on the user evaluation of teleoperation interfaces for professional service robots. In: International Conference on Human-Computer Interaction.

Shah, R., Ahmed, C.M., Nagaraja, S., 2022. Can you still see me?: Identifying robot operations over end-to-end encrypted channels. In: Proceedings of the 15th ACM Conference on Security and Privacy in Wireless and Mobile Networks.

Shen, M., Ji, K., Gao, Z., Li, Q., Zhu, L., Xu, K., 2023. Subverting Website Fingerprinting Defenses with Robust Traffic Representation. In: Proc. of USENIX Security Symposium.

Shmatikov, V., Wang, M.-H., 2006. Timing analysis in low-latency mix networks: Attacks and defenses. In: Proceedings of the 11th European Symposium on Research in Computer Security. pp. 18–33.

Sirinam, P., Imani, M., Juarez, M., Wright, M., 2018. Deep fingerprinting: Undermining website fingerprinting defenses with deep learning. In: Proceedings of the 2018 ACM SIGSAC conference on computer and communications security.

Tang, C., Barradas, D., Hengartner, U., Hu, Y., 2025a. Network traces for sawyer's joint-level controller actions. https://doi.org/10.5281/zenodo.15007304.

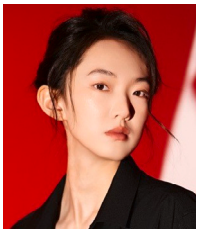
Tang, C., Barradas, D., Hengartner, U., Hu, Y., 2025b. On the feasibility of fingerprinting collaborative robot network traffic. In: International Conference on Availability, Reliability and Security. Springer, pp. 95–117.

Tang, C., Barradas, D., Hengartner, U., Hu, Y., 2025c. Uncovering robot joint-level controller actions from encrypted network traffic. In: IFIP International Conference on ICT Systems Security and Privacy Protection. Springer, pp. 121–135.

Wang, T., Cai, X., Nithyanand, R., Johnson, R., 2014. Effective attacks and provable defenses for website fingerprinting. In: Proc. of USENIX Security Symposium.

Wu, L., Alqasemi, R., Dubey, R., 2020. Development of smartphone-based human-robot interfaces for individuals with disabilities. IEEE Robot. Autom. Lett. 5 (4).

Xue, D., Ramesh, R., Jain, A., Kallitsis, M., Halderman, J.A., Crandall, J.R., Ensafi, R., 2022. OpenVPN is open to VPN fingerprinting. In: Proc. of USENIX Security Symposium.



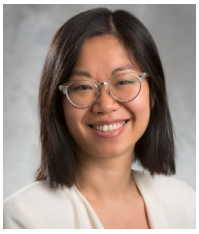
Cheng Tang is a Research Assistant at the Active & Interactive Robotics Lab at the University of Waterloo, Canada, since 2022. Her research interests are in social and companion robots, soft actuation systems, machine learning methods, cybersecurity and privacy. She has first authored several peer reviewed publications in international venues, and has obtained the Best Research Paper Award for her first authored paper at the The International Conference on Availability, Reliability and Security (ARES) 2025.



Diogo Barradas is an Assistant Professor at the Cheriton School of Computer Science at the University of Waterloo and a member of the CrySP group, serving as Associate Director of the Waterloo Cybersecurity and Privacy Institute (CPI) since March 2024. His track record counts with over 35 peer-reviewed publications, several of which included in top security (IEEE S&P, ACM CCS, USENIX Security, NDSS) and privacy (PoPETs) venues. Diogo received a 2025 FOCI Rising Star Award for his continued work in internet freedom. Earlier, he completed a Ph.D. at Instituto Superior Técnico Universidade de Lisboa and a post-doc at Rice University.



Urs Hengartner is an Associate Professor in the David R. Cheriton School of Computer Science at the University of Waterloo, Canada, where he is a founding member of the Cryptography, Security, and Privacy (CrySP) research group. His research interests are in information privacy and computer and networks security. His current research focuses on user authentication, including authenticating users based on their behavioral or physiological biometrics. He has a degree in computer science from ETH Zurich and an M.S. and Ph.D. in computer science from Carnegie Mellon.



Yue Hu is an Assistant Professor in the Department of Mechanical and Mechatronics Engineering at the University of Waterloo, where she directs the Active and Interactive Robotics (AIRLab) since 2021. She holds a Ph.D. in Computer Science from Ruprecht Karl University of Heidelberg (Germany) and a postdoc from AIST (Japan). Her research focuses on collaborative robotics, physical humanrobot interaction, and robotics cybersecurity, with recent work exploring privacy and security risks in robotic systems, including the analysis of encrypted network traffic for teleoperated robots. Dr. Hu's research integrates robotics, control, human-centered design, and cyber–physical system security to develop safer, more trustworthy robotic technologies for real-world applications.